

Secure cookie is a type of an HTTP cookie that has the Secure attribute set, which limits the scope of the cookie to "secure" channels (where "secure" is defined by the user agent, typically web browser). When a cookie has the Secure attribute, the user agent will include the cookie in an HTTP request only if the request is transmitted over a secure channel (typically HTTPS).

Although seemingly useful for protecting cookies from active network attackers, the Secure attribute protects only the cookie's confidentiality. An active network attacker can overwrite Secure cookies from an insecure channel, disrupting their integrity. This issue is officially referred to as Weak Integrity. However, some browsers, including Chrome 52 and higher and Firefox 52 and higher, forgo this specification in favor of better security and forbid insecure sites (HTTP) from setting cookies with the Secure directive.

Even with Secure, some sources recommend that sensitive information never be stored in cookies, on the premise that they are inherently insecure and this flag can't offer real protection. Secure attribute is not the only protection mechanism for cookies, there are also HttpOnly and SameSite attributes. The HttpOnly attribute restricts the cookie from being accessed by, for instance, JavaScript, while the SameSite attribute only allows the cookie to be sent to the application if the request originated from the same domain.

## Background

An HTTP cookie is a small packet of data that is sent from a web server to a user's web browser. There are two types of cookie:

Persistent cookies - Cookies that store information in the user's browser for a long time.

Non-persistent cookies - Cookies that generally expire when the browser closes.

Cookies could contain sensitive information, such as passwords and credit card numbers, which are sent over an HTTP connection and might be stored in web browsers as plain text. To prevent attackers from stealing this information, cookies can be secured with attributes.

## Cookie theft and hijacking

Various cookie hijacking techniques exist. The methods are not difficult to implement and can do significant damage to a user or an organization. Cookies that contain sensitive information such as usernames, passwords, and session identifiers can be captured using these tools once they are downloaded from a site to a web browser or accessed through a computer hard drive.

## Network threats

Cookies that are sent over unencrypted channels can be subject to eavesdropping, i.e. the contents of the cookie can be read by the attacker. These types of threats can be prevented by the use of transport layer security in servers and web browsers, although this works only if the cookies are on the network. One might also use cookies with only the sensitive information encrypted instead of the entirety of a data payload exchanged.

## End system threats

Cookies can be stolen or copied from the user, which could either reveal the information in the cookies or allow the attacker to edit the contents of the cookies and impersonate the users. This happens when a cookie, which is in the browser's end system and stored in the local drive or memory in clear text, is altered or copied from one computer to another with or without the knowledge of the user.

## Cookie harvesting

An attacker can try to impersonate a website by accepting cookies from the users. Once the attacker gets the cookies, he can use these harvested cookies for websites that accept third-party cookies. An example of this threat is the so-called Cross-Site Scripting attack, which

involves the exploitation of the vulnerabilities of a website displaying data provided by the user that has underlying malicious intent. An attacker, for instance, could embed a script in a URL he has posted in a discussion forum, message board, or email, which is then activated when the target opens the hyperlink.

See also

[HTTP cookie](#)

[Session \(computer science\)](#)

[Information security](#)

== References ==